

Design, analysis and experimental evaluation of exhaustive and greedy algorithms for the minimum cut problem

Marko Pongrac

Departamento de Eletrónica, Telecomunicações e Informática

mpong rac@ua.pt

Abstract – The minimum cut problem is a fundamental topic in graph theory with applications in network resilience, circuit partitioning, and clustering. This paper compares four algorithms for computing minimum cuts in undirected, unweighted graphs: exhaustive edge-subset enumeration, exhaustive vertex-partition enumeration, greedy edge removal, and a greedy partitioning method inspired by the Kernighan–Lin algorithm. The exhaustive approaches guarantee optimality but have exponential time complexity, making them impractical for larger graphs. The greedy methods, in contrast, run in polynomial time and use heuristic, local iterative improvements, trading optimality for scalability. All algorithms are formally analyzed, implemented in Python using NetworkX, and experimentally tested on randomly generated graphs. The results show that exhaustive methods become intractable as graph size grows, while greedy methods achieve optimal or near-optimal cuts with significantly lower computational complexity, demonstrating the trade-off between optimality and efficiency.

Keywords – graph theory, exhaustive search, greedy algorithm, heuristics, minimum cut

I. INTRODUCTION

Graphs are abstract mathematical models that represent pairwise relationships between different entities, and they have an important role in the fields of computer science and discrete mathematics. By analyzing the relationships between vertices (or nodes), it is possible to gain insight into the global structure, connectivity, and resilience of the systems represented by their corresponding graph. Formally, a graph G is defined as an ordered triple

$$G = (V(G), E(G), \tau_G)$$

consisting of a nonempty set $V(G)$ of vertices, a set $E(G)$ of edges, and an incidence function τ_G [1]. If each edge $e \in E(G)$ is associated with a real number $w(e)$, the real number is called the weight, and the graph is called a weighted graph.

The focus of this report is on the minimum cut problem of the given undirected, unweighted graph. This problem seeks to separate the vertices of the given

graph G into two disjoint sets S and T , so that the number of edges connecting the sets is minimized. Solving this problem allows us to evaluate stability and connectivity of the graph, providing insight into the resilience of the system it represents to failures or disruptions. Some of the currently known algorithms for this problem are capable of solving it in polynomial time, most notably Stoer-Wagner algorithm [2], its worst-case time complexity of $O(n^3)$ makes it impractical for large graphs, which highlights the complexity of the problem. [3]. Therefore, for graphs with larger numbers of vertices and edges, several heuristic methods have been proposed [4].

In this report, two different exhaustive search methods for the problem are implemented and analyzed, along with two greedy heuristics that run in polynomial or almost-polynomial time, but at the cost of providing approximate, rather than optimal solutions. In the end, the experimental results of all methods are compared in order to assess the trade-off between the precision of the developed algorithms and their computational complexity.

II. FORMAL PROBLEM DEFINITION

Let $G = (V, E)$ be an undirected, unweighted graph, where V is the set of vertices and E is the set of edges.

A *cut* of G is a partition of the vertex set V into two complementary, non-empty subsets S and T , such that

$$S \cup T = V, \quad S \cap T = \emptyset \quad S \neq \emptyset, \quad T \neq \emptyset.$$

The *size* of the cut (S, T) , denoted as $c(S, T)$, is the number of edges that are incident with one vertex from the subset S and the other one from the T

$$c(S, T) = |\{\{u, v\} \in E \mid u \in S, v \in T\}|.$$

A *minimum cut* of G is a cut (S^*, T^*) such that

$$c(S^*, T^*) = \min_{(S, T)} c(S, T),$$

in other words, it is a cut with smallest number of edges connecting vertices from sets S and T .

When conducting an experimental analysis of the designed solutions for the given problem, it is important to note some basic properties of the minimum cut problem:

- If the graph G is disconnected, the minimum cut has size 0.

- If the graph G has $|E| < |V| - 1$, it is necessarily disconnected and therefore has the minimum cut with size 0
- If the graph G is connected and has $|E| = |V| - 1$, it is necessarily a tree and consequently has the minimum cut of size 1
- If the graph G is connected and has $|E| > |V| - 1$, finding an optimal solution is not trivial as in the cases above.

Since we are not interested in trivial cases, most of the experiments were conducted on graphs that fall in the last category. Figure 1 shows an example of a minimum cut in the graph with 10 vertices and 10 edges.

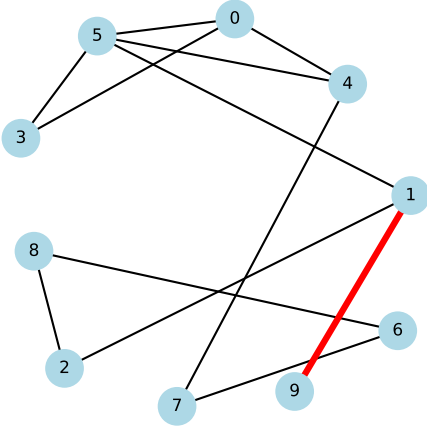


Fig. 1 - Minimum cut of graph with 10 vertices and 12 edges. The only edge that belongs to the minimum cut of the size 1 is the edge between vertices 1 and 9 (highlighted in red).

III. ALGORITHMS

A. Exhaustive Edge Subset Enumeration Algorithm

The main idea of this approach is that the original problem is equivalent to finding the smallest set of "bridge edges" whose removal disconnects the graph into two components. In essence, the algorithm enumerates all possible subsets of edges in order of increasing cardinality, and for each subset of edges checks whether the graph becomes disconnected once that subset is removed, stopping as soon as the valid cut is found. The fact that subsets are enumerated in order of increasing cardinality guarantees that the found solution will be minimum cut.

The function `check_connected` as input takes graph G and as output returns boolean value, `true` if the graph is connected, `false` otherwise. In my implementation of the function, the simple flood-fill approach (or depth-first search) was used: the algorithm starts from the node marked with 0, and tries to traverse all possible vertices in the graph. In case that visiting all vertices is not possible, we can conclude that the graph is not connected, and therefore we have found the minimum cut of the given graph.

Let $|E|$ be the number of edges in the graph G . If

Algorithm 1 Exhaustive edge subset enumeration

```

1: Input: Graph  $G = (V, E)$ 
2: Output: Minimum cut set and number of bridges
3:  $vertices \leftarrow$  vertices of  $G$ 
4:  $edges \leftarrow$  edges of  $G$ 
5:  $edge\_subsets \leftarrow generate\_edge\_subsets(edges)$ 
6:  $min\_cut \leftarrow \emptyset$ 
7:  $min\_number \leftarrow |edges|$ 
8: for each  $edges\_to\_remove$  in  $edge\_subsets$  do
9:    $current\_edges \leftarrow edges \setminus edges\_to\_remove$ 
10:   $current\_G \leftarrow$  Graph with vertices  $vertices$  and
    edges  $current\_edges$ 
11:  if not check_connected( $current\_G$ ) then
12:     $min\_number \leftarrow |edges\_to\_remove|$ 
13:     $min\_cut \leftarrow edges\_to\_remove$ 
14:    break
15:  end if
16: end for
17: return  $min\_number, min\_cut$ 

```

we are testing every possible subset of edges to remove, except the empty set and the set that contains all $e \in E(G)$, the total number of subsets to check as possible solutions is equal to $2^{|E|} - 2$. Since the number of possible subsets of edges is an exponential function of the number of edges in the graph, computational complexity increases rapidly as the size of the graph grows.

However, the number of possible subsets for the minimum cut problem can be significantly reduced by recalling the primary idea of this algorithm. Since our goal is to partition the graph into two connected components, in the worst case scenario we will isolate one vertex from the rest of the graph. Not only that we can simply isolate one vertex, but in order to achieve disconnected graph, we can isolate the vertex with the lowest degree. Let $\delta(G)$ denote the minimum degree of a vertex in the graph G . Then, an upper bound for the size of a minimum cut in G is given by

$$c(S, T) \leq \delta(G),$$

where $c(S, T)$ is the size of the minimum cut of G . Therefore, the total number of subsets to check is reduced to

$$\sum_{r=1}^{\delta(G)} \binom{|E|}{r},$$

where $|E|$ is the number of edges in the graph G . This is significantly smaller than the total number of all non-empty subsets, $2^{|E|} - 1$, especially for $\delta(G) \ll |E|$. In addition, according to [5], for the binomial coefficient, the basic bound is

$$\binom{n}{k} \leq n^k$$

Which, when summed across all numbers smaller or equal to $\delta(G)$, justifies the upper bound for total num-

ber of edge subsets to enumerate equal to

$$|E|^{\delta(G)}.$$

B. Exhaustive Partition Enumeration Algorithm

The exhaustive partition enumeration algorithm aims to find the minimum cut of the graph G by considering all possible partitions of the vertex set V into two complementary, non-empty sets S and T . For each enumerated partition, the algorithm calculates the number of edges that are incident with one vertex from the subset S and one vertex from the subset T , i.e., the edges that are bridges between the subsets S and T . The number of edges that cross the partition (S, T) is calculated by summing the appropriate entries of the adjacency matrix. Unlike enumeration of edge subsets, here we cannot end the algorithm once we find the solution, since partitions are not ordered increasingly by number of edges. Using the basic properties of the minimum cut problem, the only situation in which enumeration can be stopped early is if the number of bridges in the current best solution is equal to 1.

Algorithm 2 Exhaustive partition enumeration

```

1: Input: Graph  $G = (V, E)$ 
2: Output: Minimum cut  $(S, T)$  and number of bridges
3:  $A \leftarrow$  adjacency matrix of  $G$ 
4:  $partitions \leftarrow$  generate_vertice_partitions( $V$ )
5:  $min\_number \leftarrow |E|$ 
6:  $(best\_S, best\_T) \leftarrow (\emptyset, \emptyset)$ 
7: for each partition  $(S, T)$  in  $partitions$  do
8:    $bridges \leftarrow 0$ 
9:   for each  $u \in S$  do
10:    for each  $v \in T$  do
11:       $bridges \leftarrow bridges + A[u][v]$ 
12:    end for
13:  end for
14:  if  $bridges < min\_number$  then
15:     $min\_number \leftarrow bridges$ 
16:     $(best\_S, best\_T) \leftarrow (S, T)$ 
17:  end if
18:  if  $min\_number = 1$  then
19:    break
20:  end if
21: end for
22: return  $min\_number, best\_S, best\_T$ 
```

The correctness of this algorithm follows directly from the fact that every valid S - T cut in an undirected graph corresponds to at least one bipartition of the vertex set. Since the algorithm enumerates *all* possible bipartitions of V , the minimum cut must appear among them, and the algorithm will eventually find it. Here it is important to note that the partition (S, T) does not imply that the resulting graph is bipartite, only that the original set of vertices V is split into 2 complementary sets.

The number of generated pairs of non-empty subsets

S and T of a set of $|V|$ vertices is

$$\frac{1}{2} (2^{|V|} - 2),$$

where the factor $\frac{1}{2}$ removes symmetric duplicates, since for the minimum cut problem the pairs (S, T) and (T, S) represent the same solution. Therefore, for this algorithm the number of possible solutions to test is also the exponential function of the input size, but this time it is the exponential function of the number of vertices $|V|$ of the graph G .

C. Greedy Edge Removal

The greedy edge removal approaches the problem in the same way that exhaustive edge removal does, i.e., by removing edges to create a graph with 2 connected components. However, unlike the exhaustive approach, it does not test all possible subsets of edges. Instead, it iteratively selects the next edge to remove according to a heuristic function, updating the degrees and adjacency lists of the affected vertices. In case that one of the edges has the degree equal to 0, or the graph becomes disconnected, the algorithm stops. This approach avoids combinatorial explosion of enumerating all subsets, but comes with a price of not *always* being able to find the optimal solution.

Algorithm 3 Greedy edge removal

```

1: Input: Graph  $G = (V, E)$ 
2: Output: Minimum cut set and number of bridges
3:  $vertices \leftarrow$  vertices of  $G$ 
4:  $neighbours \leftarrow$  adjacency lists of each vertex
5:  $edges\_to\_remove \leftarrow \emptyset$ 
6:  $vertice\_degrees \leftarrow$  calculate_degrees( $neighbours$ )
7: while graph is connected do
8:    $(u, v) \leftarrow$  select_edge_by_heuristic( $vertice\_degrees$ ,  $neighbours$ )
9:    $edges\_to\_remove \leftarrow edges\_to\_remove \cup \{(u, v)\}$ 
10:  remove edge  $(u, v)$  from  $neighbours$ 
11:  update  $vertice\_degrees$ 
12:  if  $node\_levels[u] = 0$  or  $node\_levels[v] = 0$  or graph is disconnected then
13:    break
14:  end if
15: end while
16: return  $|edges\_to\_remove|, edges\_to\_remove$ 
```

The heuristic function *select_edge_by_heuristic* selects an edge to be removed by choosing vertices with the lowest current degree. First, it iterates through all vertices to find the vertex with the minimum degree. Then, among the neighbors of that vertex, it selects vertex that also has the minimum degree. The edge (vertex, neighbour) is returned as the next edge to remove. This greedy choice aims to remove edges that are “least connected” first, with the main idea being that such removals are more likely to reduce the graph to two components with fewer edge removals.

D. Greedy Partition Algorithm

The greedy partition algorithm is a heuristic approach for finding minimum cuts in undirected, unweighted graphs. Inspired by the Kernighan–Lin algorithm [6], it iteratively tries to improve an initial random partition of the graph’s vertices. Initially, the vertices are randomly divided into two sets of approximately equal size. Each vertex is assigned a gain value, representing the net change in cut size if the vertex were moved to the opposite partition.

During each iteration, the algorithm repeatedly selects the unlocked vertex with the highest gain and moves it to the other partition, updating the gains of its neighbors accordingly. Here, it is important to note that, unlike the Kernighan–Lin algorithm, which maintains the same size of each partition by swapping the pair of vertices that yields the highest gain, the greedy partition algorithm moves only one vertex at a time. Because of this, it is necessary to monitor the sizes of both partitions S and T , to avoid moving all the vertices into the same set and therefore creating an empty partition. Maintaining non-empty partitions is one of the essential conditions that the sets S and T must satisfy throughout the algorithm, because of the definition of the minimum cut problem.

Graph Partition with Node Gains

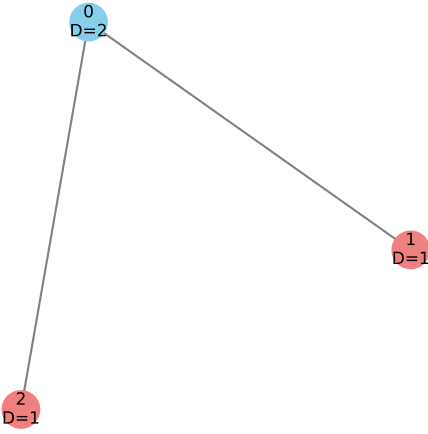


Fig. 2 - Example situation where a vertex cannot be moved without violating the non-empty partition condition, despite having the highest gain value (marked as D). Vertex 0 cannot be moved because it is the last vertex in partition S , and moving it would make S empty.

As illustrated in Figure 2, some vertices can temporarily be “locked” from moving such a move violates the non-empty set condition. This ensures that throughout the algorithm, both partitions must remain non-empty during each step so that the definition of the minimum cut problem is respected.

After every possible vertex is moved, the current cut size is tracked by updating its value after each move, and the partition corresponding to the minimal cut found in that iteration is recorded. Once all vertices are moved, a new iteration begins from the best par-

tition obtained so far. This process continues until no further improvement in cut size is observed, i.e. until the convergence of the size of the minimum cut.

While the algorithm does not guarantee that it will find an optimal solution, it operates in polynomial time and is particularly effective for larger graphs where exhaustive methods are computationally infeasible.

Algorithm 4 Greedy partition algorithm

Input: Graph $G = (V, E)$
Output: Minimum cut (S, T) and number of bridges

```

1:  $partition \leftarrow random\_split(V)$ 
2:  $nodes\_gain, cut\_size \leftarrow compute(partition, E)$ 
3:  $best\_partition \leftarrow partition, min\_cut \leftarrow cut\_size,$ 
4:  $improved \leftarrow True$ 
5: while  $improved$  do
6:    $unlocked \leftarrow V, move\_seq \leftarrow [], cut\_seq \leftarrow []$ 
7:   while  $unlocked \neq \emptyset$  do
8:     Select valid  $best\_node$  with max gain
9:     Move  $best\_node$  to other partition,
10:    Update  $S, T$  sizes
11:    Remove  $best\_node$  from  $unlocked,$ 
12:    Append  $best\_node$  to  $move\_seq$ 
13:    Update  $nodes\_gain$  of neighbors, compute
14:     $\Delta cut$ , update  $cut\_seq$ 
15:   end while
16:    $min\_idx \leftarrow \text{index of min in } cut\_seq$ 
17:   if  $cut\_seq[min\_idx] < min\_cut$  then
18:      $min\_cut \leftarrow cut\_seq[min\_idx]$ 
19:     update  $best\_partition$ 
20:      $partition \leftarrow best\_partition, \text{recompute}$ 
21:      $nodes\_gain \leftarrow compute(partition, E)$ 
22:   else
23:      $improved \leftarrow False$ 
24:   end if
25: end while
26: return  $min\_cut, best\_partition$ 

```

E. Formal Complexity Analysis

To better understand the computational requirements of the different minimum cut algorithms, their average-case and worst-case time complexities are shown in Table I. The analysis highlights the exponential growth in complexity for exhaustive methods and demonstrates how the greedy algorithms provide a more efficient alternative in practice.

As shown in Table I, exhaustive enumeration approaches suffer from combinatorial explosion, making them impractical for larger graphs. In practice, the Exhaustive Edge Subset Enumeration algorithm tends to have the highest complexity, since in most graphs the condition $|E| > |V|$ is satisfied, meaning that the number of edge subsets to be tested grows much faster than the number of vertex partitions. In contrast, the Exhaustive Partition Enumeration algorithm grows exponentially with respect to the number of vertices, rather than edges, which may still be doable for medium sized graphs.

TABLE I
TIME COMPLEXITY OF MINIMUM CUT ALGORITHMS

Algorithm	Average-case Time Complexity	Worst-case Time Complexity
Exhaustive Edge Subset Enumeration	$\Theta(E ^{\delta(G)} \cdot (V + E))$	$O(2^{ E } \cdot (V + E))$
Exhaustive Partition Enumeration	$\Theta(2^{ V -1} \cdot V ^2)$	$O(2^{ V -1} \cdot V ^2)$
Greedy Edge Removal	$\Theta(E \cdot V)$	$O(E \cdot (V ^2 + E))$
Greedy Partitioning	$\Theta(V ^2 + E)$	$O(V ^3)$

It is important to note that time complexity of the algorithms is influenced not only by configurations tested, but also by the number of inner operations. In the edge-subset algorithm, checking whether the graph remains connected after edge removal is performed by using a flood-fill algorithm, whose time complexity is $O(|V| + |E|)$. In the partition-based algorithm, the number of bridges for a given partition is calculated by summing the entries of the adjacency matrix, which requires $O(|V|^2)$ operations per partition.

In contrast to both exhaustive approaches, the greedy edge removal algorithm offers a polynomial-time solution. Although it does not guarantee an optimal solution, its time complexity is dominated by repeated updates of vertex-degrees and connectivity checks, resulting in an overall average complexity of $O(|E| \cdot |V|)$. This shows a significant improvement in complexity over the aforementioned approaches.

Like the greedy edge removal heuristic, the greedy partitioning algorithm belongs to the *PTIME* complexity class [3]. Its complexity grows more slowly with the number of edges, since it manipulates partitions rather than modifying the graph. The initial random split of vertices requires $O(|V|)$ for shuffling and labeling each vertex with either *S* or *T*. Each iteration of the improvement phase updates gain values of unlocked vertices and adjusts the cut incrementally: updating a vertex affects only its neighbors, thus having time complexity of $O(\deg(v))$, while flipping the selected vertex's gain is $O(1)$. Overall, the average-case running time is $\Theta(|V|^2 + |E|)$, and in the worst case (when $I = O(|V|)$), it reaches $O(|V|^3)$.

IV. EXPERIMENTAL METHODOLOGY

To evaluate the performance of the minimum cut algorithms, experiments were conducted on randomly generated undirected, unweighted graphs of varying sizes and densities. To enable comparison between exhaustive and heuristic algorithms, the number of vertices ranged from 3 to 24. Edge densities were set to 0.125, 0.25, 0.5, 0.75, and 1, covering both sparse and dense graph topologies. Graphs with more than 7 vertices and edge probability set to lower than 1 were generated with 5 different topologies, giving in total 375 graphs.

All algorithms were implemented in Python using the NetworkX python library [7] and executed on an 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz processor. Each algorithm was given a maximum runtime limit of three minutes per instance. Performance metrics include the size of the minimum cut found, execu-

tion time, number of basic operations performed, and number of configurations tested as possible solutions. Additionally, an important metric measured for greedy heuristic approaches was the precision metric, defined as

$$\text{Precision} = \frac{|\{G_i \mid \text{heuristic}(G_i) = \text{optimal}(G_i)\}|}{|\{G_i \mid \text{heuristic}(G_i)\}|}$$

where $\text{heuristic}(G_i)$ is the size of the minimum cut found by the heuristic algorithm, and $\text{optimal}(G_i)$ is the minimum cut size determined by the exhaustive approach.

V. RESULTS

A. Algorithm Completion Rate

On all randomly generated graphs, both greedy heuristics, as well as the exhaustive partition enumeration managed to find the solution in the set runtime limit. In contrast, the exhaustive edge subset enumeration method failed to scale to denser graphs: despite the applied optimization, it was only able to consistently solve the problem for graphs with number of edges smaller than 40. However, the optimization allowed it to handle some larger, but sparser graphs, resulting in the final completion rate of 0.658. Therefore, the largest graph that any exhaustive method managed to solve in the set time limit had 24 vertices and 276 edges.

B. Time Complexity

The relationship between the size of the input graph and empirical running time is illustrated in Figures 3 and 4, where the average number of basic operations is plotted on a logarithmic scale against the number of vertices and edges, respectively. The two greedy heuristics show only a mild increase in the number of basic operations as graph size grows. On the figures, their curves rise much slower compared to the both exhaustive algorithms, and do not form straight lines, indicating that their empirical running time is much closer to polynomial behaviour within the tested range. In contrast, both exhaustive methods exhibit near-exponential growth: partition enumeration increases rapidly with the number of vertices, while edge subset enumeration grows even faster with the number of edges, matching their theoretical worst-case bounds.

Here, it is important to note that during the evaluation of the average number of basic operations, only algorithm runs that managed to find the solution under the set runtime limit were taken into consideration. Therefore, exhaustive edge-subset enumeration

appears to outperform exhaustive partition enumeration on sparse graphs with many vertices, but this advantage is misleading: as shown in Figure 4, the edge-subset method fails completely once the graph exceeds roughly 60 edges, and already begins to time out frequently beyond 40 edges. In contrast, the exhaustive partition enumeration algorithm continues to operate on much denser graphs, demonstrating that it is more resilient to increases in edge count, even though its complexity grows exponentially with the number of vertices

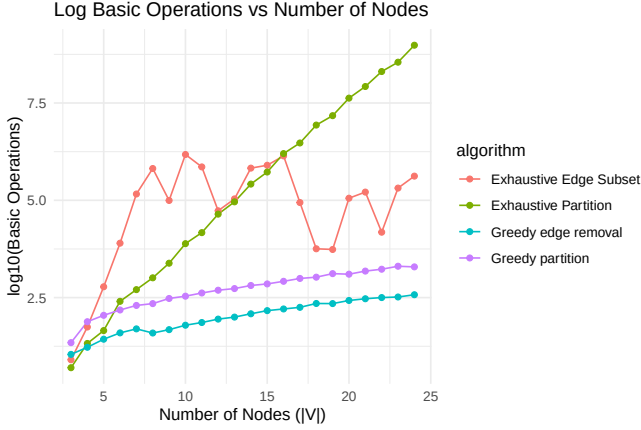


Fig. 3 - Logarithm of the average number of basic operations as a function of the number of vertices.

Both greedy heuristics show near-linear growth in the number of operations, confirming their practical efficiency even on larger graphs. In contrast, the two exhaustive methods exhibit different scaling behaviour: exhaustive partition enumeration grows roughly exponentially in the number of vertices, while exhaustive edge subset enumeration grows exponentially in the number of edges. This difference explains why the former remains feasible for graphs with up to 24 vertices, while the latter fails much earlier on denser instances.

The plots clearly show that once the graph size exceeds a certain threshold, the curves for the exhaustive algorithms rise sharply, diverging from the slower-growing heuristic curves by several orders of magnitude. This gap reflects the combinatorial explosion predicted by their theoretical worst-case bounds and is consistent with the observed completion rates described in the previous section.

C. Execution time

Like the number of basic operations, the execution time of each algorithm was also measured. Figure 5 shows the relationship between the number of vertices in the graph and needed running time to find the minimum cut. As we can see, the average running time for the exhaustive partition scales linearly on the log scale with the number of nodes, while both heuristic methods do not display strong correlation with the increasing number of vertices.

Based on the observed values, we can conclude that the average running time of the exhaustive partition

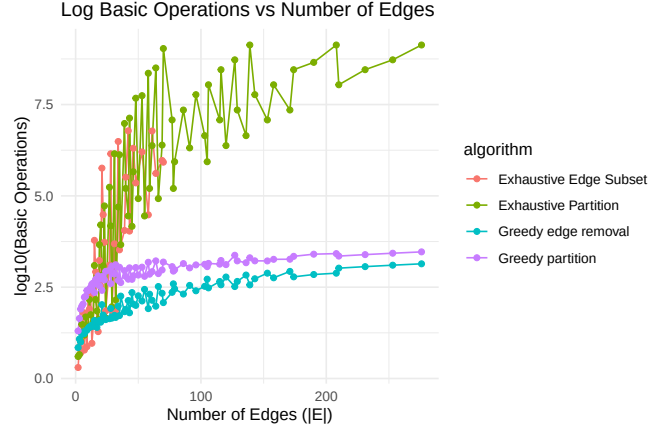


Fig. 4 - Logarithm of the average number of basic operations as a function of the number of edges.

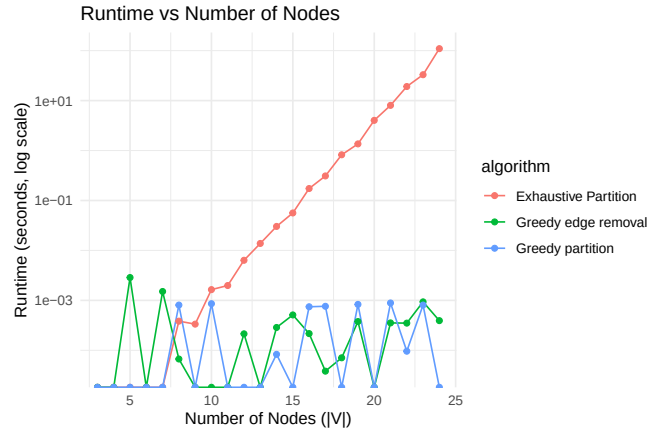


Fig. 5 - Logarithm of the running time in seconds in relation to the number of nodes in the graph

grows exponentially in relation to the number of nodes in the graph, which confirms previous hypothesis. In other words, since the average running time for the graph with 24 nodes was 125 seconds and for the graph with 23 nodes was 33 seconds, predicted average running time for the graph with 25 nodes would be approximately 478 seconds, or a little bit less than 8 minutes.

D. Number of Solutions Tested

Table II shows the Pearson correlation between the number of configurations tested and the number of basic operations performed by each algorithm. As expected, both exhaustive methods exhibit a near-perfect positive correlation, since their runtime is directly determined by the number of explored configurations. The greedy edge removal heuristic also shows a strong correlation, although slightly lower due to pruning effects and early termination. In contrast, the greedy partitioning heuristic displays a noticeably weaker correlation, confirming that it does not explore a fixed number of configurations per graph and is therefore less sensitive to graph size in terms of search space, since number of basic operations is directly connected to the size of the graph.

TABLE II
PEARSON CORRELATION COEFFICIENT BETWEEN THE NUMBER OF
CONFIGURATIONS TESTED AND THE NUMBER OF BASIC
OPERATIONS

Algorithm	Samples	Correlation
Exhaustive Edge Subset	247	0.973
Exhaustive Partition	375	0.999
Greedy Edge Removal	375	0.967
Greedy Partitioning	375	0.797

E. Heuristic Precision

The precision of both greedy heuristics was evaluated by comparing the size of the minimum cut they returned with the optimal value obtained using the exhaustive partition enumeration algorithm. Table III summarizes the results. The greedy edge removal heuristic achieved a perfect match rate of 100%, which implies both **MAE** (Mean Absolute Error) and **MaxAE** (Maximum Absolute Error) equal to 0. In contrast, the greedy partitioning heuristic deviated from the optimal solution in a small number of cases, resulting in a mean absolute error of 0.085 and a maximum deviation of 3 cut edges, while still correctly solving 93.1% of the instances.

TABLE III
PRECISION OF GREEDY HEURISTICS COMPARED TO EXACT
MINIMUM CUT

Algorithm	MAE	MaxAE	Precision
Greedy Edge Removal	0.0	0.0	1.0
Greedy Partitioning	0.085	3.0	0.931

Figure 6 shows the distribution of the non-zero absolute errors produced by the greedy partition algorithm, illustrating how frequently each error magnitude occurs. Most graphs produce very small errors, overestimating minimum cut by only 1 edge, while higher overestimation by 2 or more edges is much less common. Figure 7 shows an example of a graph for which

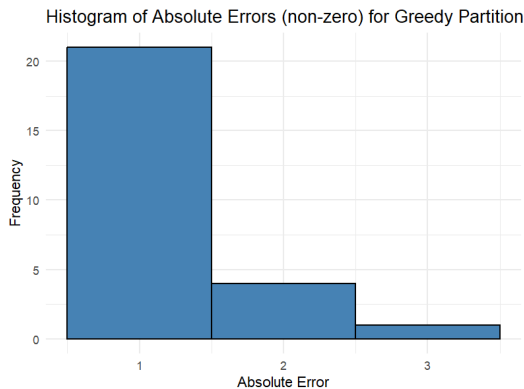


Fig. 6 - Histogram of the frequencies of non-zero absolute errors for the greedy partition algorithm.

the greedy partition algorithm had the maximum absolute error. The edges highlighted in red belong to the minimum cut set with size 4, while the optimal

solution contains only 1 edge, with possible solution being any of edges (0, 17), (1, 13), (3, 9), (5, 11) and (11, 18), since they are leafs of the observed graph.

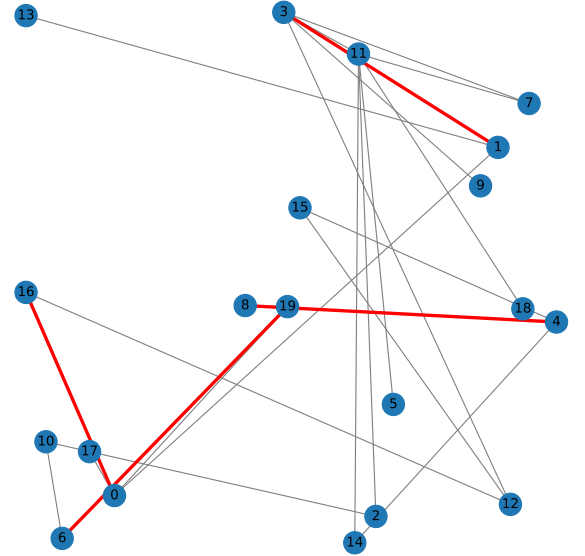


Fig. 7 - Example of a graph with highlighted minimum cut edges determined by greedy graph partition

VI. DISCUSSION

The results demonstrate a clear separation in scalability between exhaustive and heuristic methods. While exhaustive methods guarantee optimal solution, they proved to be intractable even for medium sized graphs, with exhaustive edge removal not being able to perform consistently on graphs with more than 40 edges, the number of edges that becomes more common in graphs with at least 10 vertices.

On the other hand, both greedy heuristics showed significant improvement in time complexity for the given problem, with minimal drop in precision. Unlike the exhaustive methods, their performance demonstrated polynomial time complexity and polynomial relationship between number of configurations tested and number of nodes in the graph. While greedy edge removal showed slightly better performance across most important metrics, precision and number of basic operations, it still cannot solve some graphs, for example clusters of complete graphs connected by a single edge. Figure 8 shows an example where greedy edge removal fails to "detect" that the optimal solution is simply to cut the connection between vertices 0 and 10. Instead, the algorithm "concluded" that the optimal solution is isolating one of the edges in any cluster, among those not connected to the other cluster.

In contrast, the greedy partitioning algorithm reasons over sets of vertices rather than individual vertex degrees, and consequently is less constrained to purely local decisions and can recognize that a single inter-cluster edge forms the true minimum cut, as in the

forementioned example. While this makes it computationally slightly more expensive and the current implementation still fails on some specific test graphs, its design suggests that with some additional refinements, for example, by randomly relocating vertices between partitions with a fixed probability—may allow it to escape locally optimal solutions and recover the global solution more consistently.

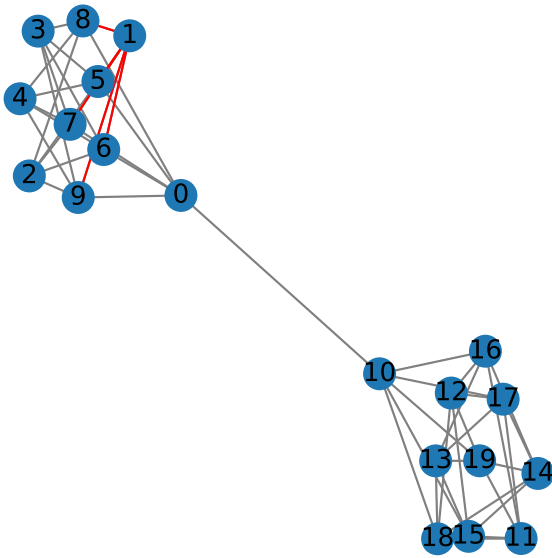


Fig. 8 - Example of a graph consisting of two connected $K_{5,5}$ clusters, with the minimum cut returned by the greedy edge removal algorithm highlighted in red.

VII. CONCLUSION

This paper evaluated four algorithms for finding the minimum cut in undirected, unweighted graphs, analyzing implementations of two exhaustive methods and two heuristic methods. The experimental results confirmed the expected exponential scaling of exhaustive approaches, which makes them impractical beyond small graph sizes despite guaranteeing optimality. In contrast, both greedy heuristics achieved dramatically better scalability, with polynomial or near-polynomial growth in time complexity and only a small loss in precision, demonstrating that approximate methods can offer a viable alternative to the intractable problems.

Among the heuristics, greedy edge removal was the fastest and most accurate on average, but also the least structurally aware, failing on an example of a graph where the minimum cut is defined by a single weak inter-cluster connection. The greedy partitioning heuristic, while slightly slower, showed greater potential to capture higher-order graph structure and could be extended to handle such difficult cases. This indicates that there is a meaningful trade-off between computational complexity and expressive power even within different heuristic methods.

The limitation of this work is that all experiments

were conducted on randomly generated graphs rather than on publicly available datasets that reflect real-world problems. Therefore, the applicability of the developed algorithms to practical domains remains uncertain, and additional validation on representative real-world graphs is required before stronger conclusions can be drawn.

Future work should therefore focus on (1) improving greedy partition algorithm to avoid local optimums, for example by applying the genetic algorithm [8] and evolutionary computation concepts, (2) testing the developed algorithms on real-world graph structures.

Overall, the results show that exhaustive methods are intractable, while well-designed heuristic strategies can find optimal solutions with significantly reduced computational complexity, offering a viable alternative to exhaustive methods on larger, more complex graphs.

REFERENCES

- [1] J. A. Bondy and U. S. R. Murty, *Graph theory with applications*, Macmillan London, 1st edition, 2008.
- [2] Mechthild Stoer and Frank Wagner, “A simple min-cut algorithm”, *J. ACM*, vol. 44, no. 4, pp. 585–591, July 1997.
URL: <https://doi.org/10.1145/263867.263872>
- [3] Michael R. Garey and David S. Johnson, *Computers and Intractability; A Guide to the Theory of NP-Completeness*, W. H. Freeman & Co., USA, 1990.
- [4] Monika Henzinger, Alexander Noe, Christian Schulz, and Darren Strash, “Practical minimum cut algorithms”, *ACM J. Exp. Algorithmics*, vol. 23, Oct. 2018.
URL: <https://doi.org/10.1145/3274662>
- [5] Matoušek Jiří and Vondrák Jan “The probabilistic method: Lecture notes”, Tech. Rep., Department of Applied Mathematics, Charles University, Malostranské nám. 25, 118 00 Praha 1, Czech Republic, 2008, Lectures revised March 2008. Available at <https://www.cs.cmu.edu/~15850/handouts/matousek-vondrak-prob-ln.pdf>.
- [6] B. W. Kernighan and S. Lin, “An efficient heuristic procedure for partitioning graphs”, *The Bell System Technical Journal*, vol. 49, no. 2, pp. 291–307, 1970.
- [7] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart, “Exploring network structure, dynamics, and function using networkx”, in *Proceedings of the 7th Python in Science Conference (SciPy2008)*, Gäel Varoquaux, Travis Vaught, and Jarrod Millman, Eds., Pasadena, CA, USA, August 2008, pp. 11–15.
- [8] David E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley Longman Publishing Co., Inc., USA, 1st edition, 1989.